

Documentation Projet BDD:

Analyse:

Propriétés :

{ EmailUser, NomUser, PrénomUser, AdressePostale,
IdProduit, NomProduit, PrixDeRevient, Stock,
NomCat, DescriptionCat,
IdSalle,
IdVente, PrixDépart, TypeVente, LimiteOffre, DateHeureDébutVente, Durée,
VenteRévocable,
PrixOffreProposé, DateHeureDépôtOffre, Quantité Produit,
NomCaractéristique, ValeurCaractéristique }

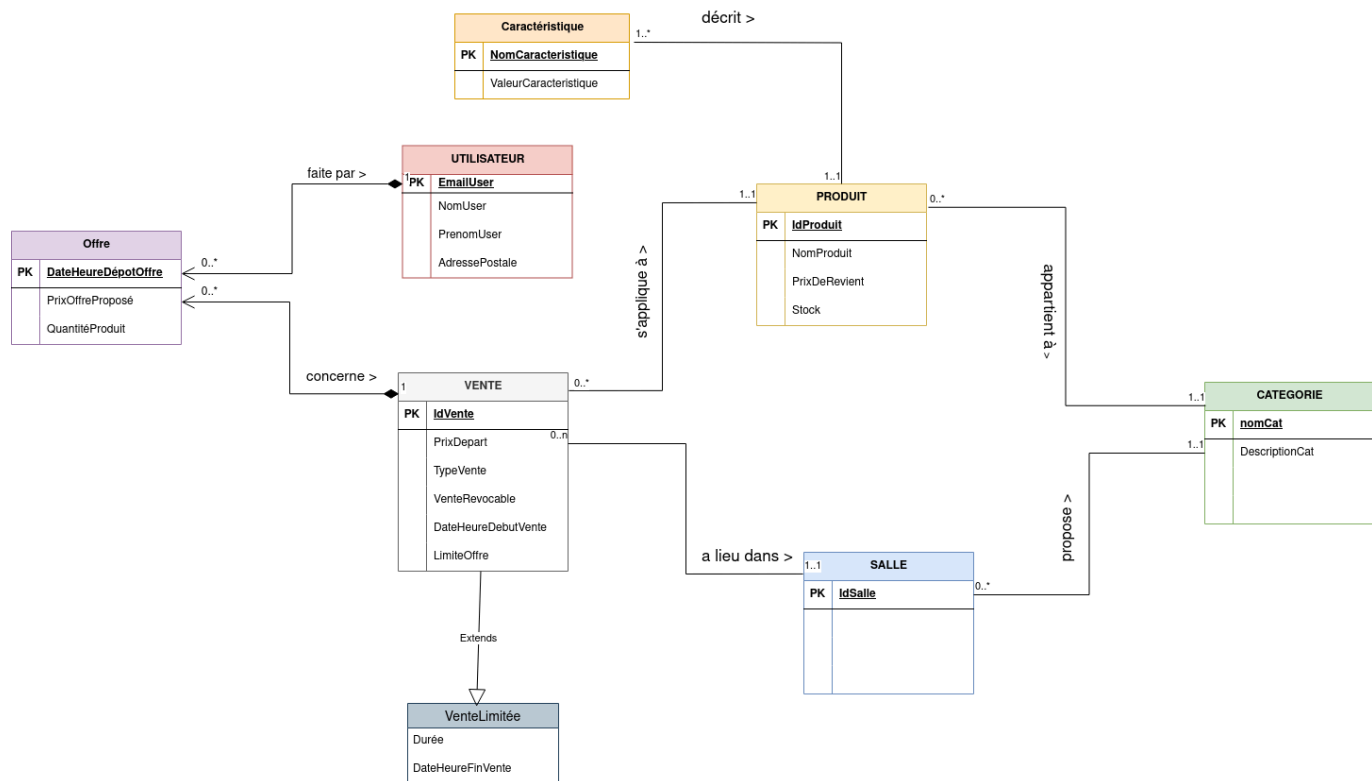
Contraintes :

Dépendances fonctionnelles	Contraintes de valeurs	Contraintes de Multiplicité
- EmailUser → NomUser, PrénomUser, Adresse Postale - Id Produit → NomProduit, PrixDeRevient, stock, NomCat - NomCat → DescriptionCat - IdSalle → NomCat - IdVente → IdProduit, IdSalle, TypeVente, Prix Départ, LimitéOffre, DateHeureDebutVente, DateHeureFinVente, Durée, VenteRevocable - EmailUser, DateHeureDepotOffre, Id Vente → PrixOffreProposé, DateHeureDepotOffre, QuantitéProduit - NomCaractéristique → ValeurCaractéristique, IdProduit	- Ventes : PrixDépart > 0 - Produit : PrixRevient > 0 - Produit : Stock > 0 - Offre : QuantitéProduit > 0 - Vente : Durée > 0 ou ILLIMITÉ	- nom Cat - ->> Id Salle - nomCat - ->> IdProduit - IdProduit -->> Nom Caractéristique - EmailUser - ->> DateHeureDepotOffre - IdVente - ->> DateHeureDepotOffre - IdProduit - ->> IdVente

Contraintes Contextuelles

- Une vente est annulée si le prix de revient n'est pas atteint (pour les ventes révocables).
- Dans les ventes descendantes, le premier utilisateur à faire une offre sur le lot complet remporte la vente.
- La vente est décidée sur le prix individuel d'un produit (si un utilisateur propose d'acheter 20 pièces à 19e et un autre d'acheter 1 pièce à 1e on prend le deuxième)

Conception du Schéma Entités/Associations:



- On a choisi de considérer OFFRE comme une entité faible par rapport à UTILISATEUR et VENTE, puisque OFFRE ne possède pas suffisamment de propriétés pour s'identifier elle-même
- On a mis VENTE LIMITÉE comme un sous-type de l'entité VENTE au lieu d'ajouter dans Vente une propriété Vente Limitée (vrai ou faux) et une propriété Durée, car celle-ci risque d'être Null si la vente est illimitée (dans ce cas Vente ne serait même pas 1FN!)

Contraintes non présentées:

Contraintes de valeurs:

- Ventes : PrixDépart > 0
- Produit : PrixRevient > 0
- Produit : Stock > 0
- Offre : QuantitéProduit > 0
- Vente : Durée > 0 ou ILLIMITÉ

Contraintes Contextuelles:

- Une vente est annulée si le prix de revient n'est pas atteint (pour les ventes révocables).
- Dans les ventes descendantes, le premier utilisateur à faire une offre sur le lot complet remporte la vente.

- La vente est décidée sur le prix individuel d'un produit (si un utilisateur propose d'acheter 20 pièces à 19e et un autre d'acheter 1 pièce à 1e on prend le deuxième)

Traduction en relationnel:

Le passage au relationnel a été assez fluide en suivant les règles du cours

- **UTILISATEUR** {EmailUser{pk}, NomUser, PrenomUser, Adresse Postale}
- **VENTE** {Id Vente{pk}, IdProduit{fk}, IdSalle{fk}, Prix Départ, Type Vente, Vente Révocable, DateHeureDébutVente, LimiteOffre}

→ Salle provient de SALLE, IdProduit provient de PRODUIT

- **VENTELIMITÉE** {IdVente{fk}, Durée, DateHeureFinVente}

→ Id Vente provient de VENTE

- **OFFRE** { DateHeureDépotOffre{pk}, EmailUser{fk}, Id Vente{fk}, PrixOffreProposé, QuantitéProduit}

→ Email User provient de UTILISATEUR, Id Vente provient de VENTE

- **CATÉGORIE** {nomCat{pk}, DescriptionCat}
- **PRODUIT** {IdProduit{pk}, NomCat{fk}, NomProduit, PrixRevient, Stock}

→ Id Vente provient de VENTE, nom Cat provient de CATÉGORIE

- **SALLE** {Salle{pk}, nom Cat{fk}}

→ nom Cat provient de CATÉGORIE

- **CARACTÉRISTIQUE** {nom Caractéristique{pk}, IdProduit{fk}, valeur Caractéristique}

→ IdProduit provient de PRODUIT

Contraintes non présentées:

Contraintes de valeurs:

- Ventes : PrixDépart > 0
- Produit : PrixRevient > 0
- Produit : Stock > 0
- Offre : QuantitéProduit > 0
- Vente : Durée > 0 ou ILLIMITÉ

Contraintes Contextuelles:

- Une vente est annulée si le prix de revient n'est pas atteint (pour les ventes révocables).
- Dans les ventes descendantes, le premier utilisateur à faire une offre sur le lot complet remporte la vente.
- La vente est décidée sur le prix individuel d'un produit (si un utilisateur propose d'acheter 20 pièces à 19€ et un autre d'acheter 1 pièce à 1€ on prend le deuxième)

Remarque : Un produit est décrit par au moins une caractéristique

Formes Normales:

Pour chaque relation R des relations ci-dessus:

- tout attribut ne contient que des valeurs atomiques et non Null (1FN)
- tous les attributs non clés sont pleinement dépendants de chacune des clefs (2FN)
- tout attribut non clé ne dépend pas d'un ensemble d'attributs qui n'est pas une clef (3FN)
- pour toute DF ($X \rightarrow Y$) non triviale (i.e. $Y \not\subseteq X$), X contient une clef de R (3FNBCK)

Toutes les relations sont 3FNBCK

Analyse des fonctionnalités:

- ★ La mise en place d'une Salle de Vente et la sélection de produits déjà disponibles à la vente et permettant le choix du type d'enchères et du prix de départ

```
INSERT INTO SALLE (IdSalle, nomCat) VALUES (7, 'electronique');
```

```
INSERT INTO VENTE (IdVente, IdSalle, IdProduit, PrixDepart, TypeVente,
VenteRevocable, DateHeureDebutVente, LimiteOffre)
VALUES (10, 7, 1, 600.00, 'croissante', 0, SYSTIMESTAMP, 0);
```

```
COMMIT;
```

- ★ L'enchère faite par un utilisateur sur un produit mis en vente dans une Salle de Vente

```
-- Vérification de validité de l'offre
```

```
SELECT 1
```

```
FROM VENTE v
```

```
JOIN OFFRE o ON v.IdVente = o.IdVente
```

```
WHERE v.IdVente = 10
```

```
AND (
```

```
    (v.TypeVente = 'croissante' AND 700.00 > (SELECT MAX(PrixOffrePropose)
```

```
FROM OFFRE WHERE IdVente = 10))
```

```
    OR (v.TypeVente = 'decroissante' AND 700.00 < (SELECT
```

```
MIN(PrixOffrePropose) FROM OFFRE WHERE IdVente = 10))
```

```
);

-- Insertion de l'offre si elle est valide
INSERT INTO OFFRE (DateHeureDepotOffre, EmailUser, IdVente,
PrixOffrePropose, QuantiteProduit)
VALUES (SYSDATE, 'eve@example.com', 10, 700.00, 1);

COMMIT;
```

- ★ Le processus de fin d'enchère déterminant le (ou les) utilisateur(s) ayant remporté une vente, en tenant compte du type d'enchères bien sûr

```
SELECT EmailUser, PrixOffrePropose, QuantiteProduit
FROM OFFRE
WHERE IdVente = 10
AND (
    (SELECT TypeVente FROM VENTE WHERE IdVente = 10) = 'croissante'
    AND PrixOffrePropose = (SELECT MAX(PrixOffrePropose) FROM OFFRE WHERE
    IdVente = 10)
    OR
    (SELECT TypeVente FROM VENTE WHERE IdVente = 10) = 'decroissante'
)
ORDER BY CASE WHEN (SELECT TypeVente FROM VENTE WHERE IdVente =
10) = 'croissante' THEN PrixOffrePropose END DESC,
    CASE WHEN (SELECT TypeVente FROM VENTE WHERE IdVente = 10) =
'descendante' THEN DateHeureDepotOffre END ASC
FETCH FIRST 1 ROWS ONLY;
```

Bilan:

- Bonne organisation, on a travaillé en parallèle la création des relations d'une part et l'implémentation du démonstrateur d'autre part. Les tâches étaient claires et bien réparties au sein du groupe
- Un point difficile qu'on a rencontré était la modélisation de l'entité Offre (s'il fallait introduire un nouvel Id pour Offre, si c'était acceptable de la considérer comme une entité faible par rapport à deux autres entités ou faire une association ternaire...)

Mode d'emploi du démonstrateur:

Pour tout compiler: **make all**

Pour lancer l'application: **make exeSimuGraphique**

